Artefato 5 – Descrição da Arquitetura do Software

Architecture Notebook

Sistema de apoio à gestão de requisitos com histórias de usuário

Este documento descreve **como o sistema é organizado internamente**: seus componentes, relações, decisões, mecanismos, abstrações e perspectivas. Distingue mecanismos presentes no código de requisitos de implantação, sem detalhar wireframes, tabelas físicas ou infraestrutura.

Identificação	Descrição
Documento	Artefato 5 – Architecture Notebook.
Sistema	Aplicação web colaborativa para gerir projetos, Product Backlogs, Sprint Backlogs, histórias, critérios de aceitação, épicos e priorização.
Tecnologias de referência	Python, Django, mecanismo de templates do Django e SQLite.
Escopo	Requisitos e dependências de arquitetura; decisões e restrições; componentes e relacionamentos; mecanismos, abstrações, perspectivas e impacto das ferramentas.

1 Objetivos e suposições da arquitetura

Objetivos. A arquitetura deve (1) organizar requisitos por projeto e seus backlogs; (2) representar histórias, épicos e múltiplos critérios de aceitação; (3) permitir colaboração entre contas com papéis de dono e membro e autorização por projeto; (4) separar Story Points, usados no planejamento das Sprints, de Effort, empregado no RICE para priorizar o Product Backlog; (5) separar apresentação, tratamento de requisições, domínio e persistência; e (6) manter relações consistentes em criação, movimentação e exclusão de registros.

Suposições. Segundo os artefatos, o público inclui grupos pequenos de usuários de 2 a 8 pessoas por projeto, com interface web planejada em português. A solução é uma aplicação Django única, com persistência em SQLite, executável localmente para desenvolvimento e em servidor acessível por HTTPS para uso compartilhado. Cada projeto possui um dono e pode ter membros de outras contas. Não há edição colaborativa em tempo real nem serviço de envio de e-mails: o dono adiciona uma conta já cadastrada pelo endereço de e-mail.

2 Dependências e requisitos relativos à arquitetura

Dependências tecnológicas. O funcionamento utiliza Python, Django, Django Templates, ORM, autenticação/sessões e SQLite; o cliente precisa de navegador web. No repositório, `projeto_engsoftware` contém configuração e rotas, e `core` contém o domínio. No ambiente de produção definido, Nginx recebe HTTPS e encaminha requisições a Gunicorn, que executa Django. Tais componentes são dependências de implantação, não novas camadas de domínio. Nenhum serviço externo de e-mail ou API integra o fluxo essencial.

ID	Requisito/condicionante arquitetural	Consequência para a estrutura
RA01	Separar interface, tratamento de requisições, regras do domínio e persistência.	Utilizar a organização MVT do Django e acesso a dados mediado pelo ORM.
RA02	Permitir compartilhamento de projetos, distinguindo dono e membros (HU04–HU07, HU22–HU24).	Identidade Django, vínculo usuário–projeto e autorização por operação: só o dono altera/exclui o projeto e gerencia membros.
RA03	Um Product Backlog é criado junto ao projeto; há múltiplas Sprints e cada história aparece em um único backlog por vez (HU04, HU11).	Relações do domínio e regra de transferência consistente de histórias entre backlogs do mesmo projeto.
RA04	Histórias usam Como/Eu quero/Para e podem conter vários critérios Dado/Quando/Então, editáveis e excluíveis individualmente (HU12–HU13).	Abstração de critério vinculada à história e validação das partes obrigatórias.

ID	Requisito/condicionante arquitetural	Consequência para a estrutura
RA05	Separar Story Points de Effort e aplicar escalas de entrada, MoSCoW e RICE (HU18–HU21).	Campos independentes; RICE incompleto fica sem score numérico e Effort zero gera valor não calculável.
RA06	Proteger dados de projetos compartilhados e preservar integridade em operações HTTP.	Middleware CSRF, sessões, verificações de participação no servidor e vínculos consistentes no ORM.
RA07	Favorecer manutenibilidade e alterações incrementais.	Organização modular de configuração, domínio, views e templates, com migrações versionadas.
RA08	Permitir execução local e acesso compartilhado pela internet, como previsto no Artefato 9.	Navegador/HTML; Django em desenvolvimento e HTTPS via Nginx/Gunicorn em produção, preservando o armazenamento SQLite.

Os requisitos acima são apresentados **do ponto de vista de seus efeitos sobre a estrutura**.

3 Estilo arquitetural, elementos e relacionamentos

A arquitetura adotada é a de uma **aplicação web única organizada segundo MVT (Model–View–Template)**. O Django recebe requisições HTTP, aplica os middlewares e identifica a rota. A view coordena a operação; os models representam entidades e comportamentos de domínio; o ORM acessa o SQLite; templates podem compor as respostas HTML. O sistema não requer API externa ou microsserviços para cumprir seu objetivo.

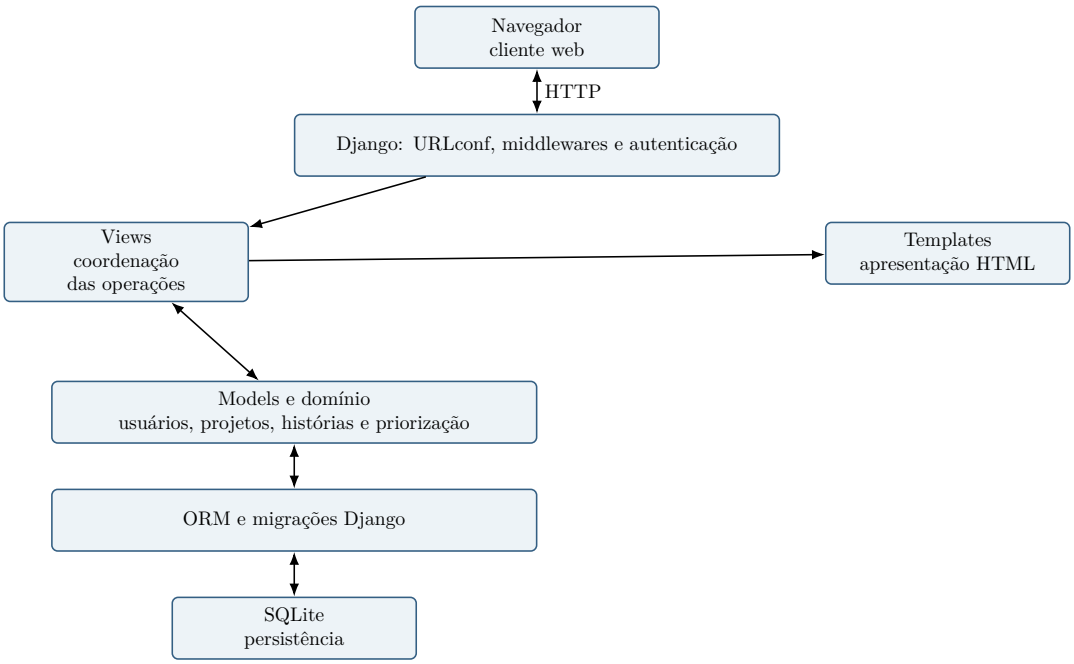


Figura 1 – Relações entre os componentes na organização MVT adotada.

Elemento	Responsabilidade	Referência no repositório
Configuração e rotas	Habilitar componentes, middlewares e associação entre URL e view.	`settings.py`, `urls.py`.
Views	Receber requisições e coordenar operações de aplicação.	`core/views.py`, `core/forms.py`.
Templates	Gerar HTML a partir dos dados entregues pelas views.	`core/templates/core/`.
Models e domínio	Definir entidades, relacionamentos e cálculo de RICE.	`core/models.py`.

Elemento	Responsabilidade	Referência no repositório
Persistência	Traduzir operações sobre objetos em consultas ao banco.	ORM, migrations e SQLite.
Identidade e administração	Identificar usuários e manter sessões; o vínculo com projetos é uma responsabilidade da aplicação.	Django Auth; `Projeto.dono` e `Projeto.membros`.

A Figura 1 é uma **visão da estrutura arquitetural**. Em produção, o percurso anterior ao Django inclui HTTPS, Nginx e Gunicorn. O URLconf também registra rotas para Sprint Backlogs, histórias de usuário, critérios de aceitação e épicos. As views correspondentes coordenam consultas, criação, edição, exclusão e movimentação, conforme as operações previstas.

4 Abstrações do domínio e perspectiva estrutural

O modelo de domínio em `core/models.py` contém seis classes principais: `Projeto`, `ProductBacklog`, `SprintBacklog`, `Epico`, `HistoriaUsuario` e `CriterioAceitacao`. **Projeto** organiza os requisitos; **Product Backlog** agrupa histórias do produto; **Sprint Backlog** representa histórias selecionadas para uma iteração; **Épico** associa histórias relacionadas; e **História de Usuário** reúne texto, vínculos, estimativa e priorização. **Critério de Aceitação** mantém múltiplas condições Dado/Quando/Então associadas à história.

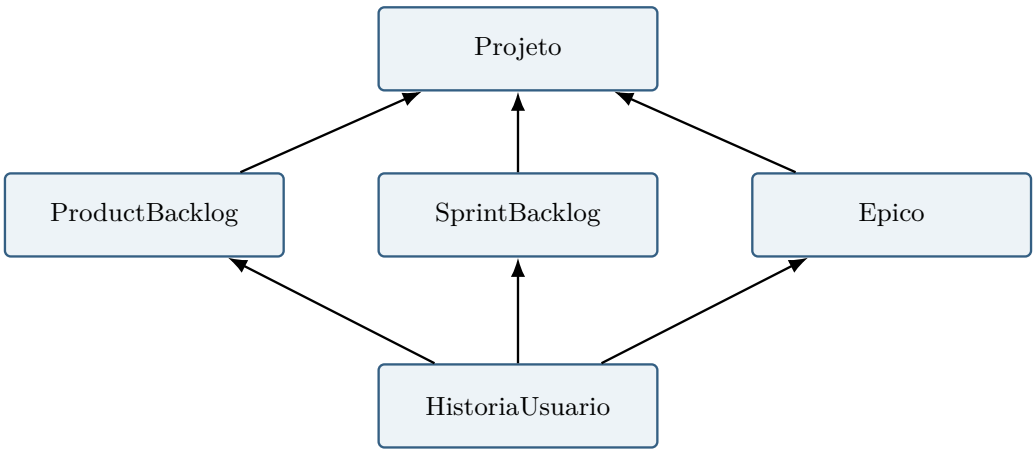


Figura 2 – Núcleo de projetos e backlogs; a relação com CriterioAceitacao é descrita no texto.

Multiplicidades e invariantes. Cada `ProductBacklog` referencia um `Projeto` via `OneToOneField`; Sprint Backlogs e épicos referenciam o projeto via `ForeignKey`, enquanto critérios de aceitação referenciam a história por `ForeignKey`. Sprints possuem datas de início e fim, validadas no modelo e por restrição de período no banco. O Artefato 4 (HU04) determina que a criação do projeto produza seu único Product Backlog. A HU11 determina que cada história apareça em um só backlog por vez. A integridade é reforçada por validações no modelo, restrição de exclusividade de backlog e gatilhos SQLite que impedem vínculos inconsistentes entre projetos. O Product Backlog pode ser excluído apenas se o projeto estiver sem histórias; a exclusão de Sprint devolve suas histórias ao Product Backlog, desde que ele exista. As transferências de histórias devem atualizar os campos de backlog conjuntamente.

Colaboração, dono e membro. Django Auth identifica contas; um vínculo com o projeto distingue *dono* e *membro*. Quem cria o projeto é seu dono; só ele altera nome/descrição, exclui projeto ou adiciona/remove membros (HU04–HU07, HU22–HU24). Dono e membros podem operar backlogs, histórias, épicos e prioridades, e um membro pode sair do projeto sem excluir os requisitos. As views verificam no servidor o vínculo do usuário com o projeto e distinguem as operações exclusivas do dono das permitidas aos membros. O modelo `Projeto` contém as relações `dono` e `membros`.

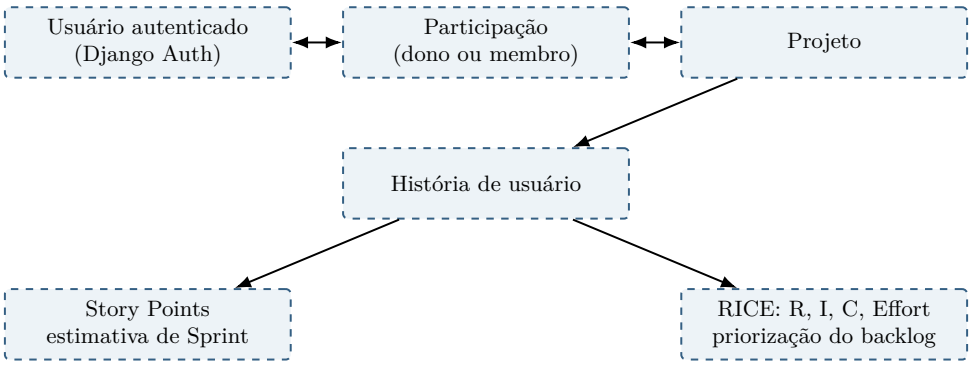


Figura 3 – Participação por projeto e separação entre Story Points e os componentes RICE implementados.

História e critério de aceitação. A história mantém `papel`, `acao` e `beneficio` (Como/Eu quero/Para). O Artefato 4 (HU12–HU13) permite adicionar, editar e remover vários critérios Dado/Quando/Então ligados a uma história. Portanto, *critério de aceitação* é uma abstração própria, com vínculo um-para-muitos com a história. A implementação contém a classe `CriterioAceitacao` com os campos `dado`, `quando` e `entao` e uma chave estrangeira para `HistoriaUsuario`. Os campos antigos `contexto`, `evento` e `resultado` permanecem no modelo para compatibilidade com registros legados; a migração 0007 transfere conjuntos completos para a nova entidade.

Estimativa e prioridade. Story Points estimam relativamente o tamanho da história para planejamento de Sprints; *Effort* é uma entrada independente do RICE para priorização do Product Backlog (HU18, HU20–HU21). Ambos seguem a escala do enunciado: 0, 1, 2, 3, 5, 8, 13, 21, 34 e 55. Com os quatro campos preenchidos e $E > 0$, o score é $(R \times I \times C)/E$, utilizando E = Effort e Confidence na escala percentual 100/80/50 indicada na especificação. Se faltar algum dado, o score é *incompleto*; para $E = 0$, é *não calculável*. Na implementação analisada, `rice_score` utiliza `effort` como divisor, separado de `story_points`. A propriedade `rice_display` distingue RICE incompleto de Effort zero (não calculável), e o valor numérico é calculado somente quando os quatro componentes estão presentes e Effort é maior que zero.

5 Mecanismos arquiteturais

Mecanismo	Finalidade e comportamento	Base tecnológica
Roteamento HTTP	Associa URLs às views de autenticação, projetos, membros, Product e Sprint Backlogs, histórias, critérios de aceitação, épicos e administração.	URLconf Django.
Autenticação e sessões	Django Auth e SessionMiddleware identificam cada conta; o mesmo projeto pode ter participantes distintos conforme os vínculos definidos na aplicação.	Django Auth.
Proteção e autorização	Middleware CSRF protege formulários; views devem testar vínculo e papel do usuário, inclusive em URL acessada diretamente. Só o dono gerencia dados do projeto e membros.	Django middleware e domínio.
Mapeamento objeto-relacional	Classes Django mapeiam entidades e relações; operações persistentes passam pelo ORM.	Models e ORM.
Relações e exclusões	ORM e operações atômicas preservam vínculos; a exclusão de Product Backlog com histórias é bloqueada. Ao excluir uma Sprint, suas histórias são transferidas ao Product Backlog; a operação é bloqueada se ele estiver ausente.	ORM e regra de domínio.
Validação de entradas	Há validações de período das Sprints, opções de Story Points e Effort independentes, MoSCoW, Impact e Confidence, complementadas por validações do domínio e restrições do banco.	Campos e regras Django.
Cálculo RICE	O código calcula $(R \times I \times C)/E$ usando o campo `effort`; campos ausentes produzem RICE incompleto e Effort zero produz não calculável.	Regra do domínio Python.

Mecanismo	Finalidade e comportamento	Base tecnológica
Movimentação de histórias	Validações e restrições impedem dois backlogs simultâneos e vínculos entre projetos diferentes. A transferência exige atualização conjunta dos vínculos.	Regra de domínio e ORM.
Evolução do esquema	Migrações registram mudanças na estrutura relacional modelada pelas classes Django.	Django migrations.

6 Decisões, restrições e justificativas arquiteturais

Decisão/restrição	Justificativa	Consequência
Aplicação web única em Django	Domínio coeso e escopo acadêmico definidos na Visão e Escopo.	Configuração, domínio e processamento convivem no mesmo sistema.
Organização MVT	Estrutura nativa do framework e separação de responsabilidades.	Views coordenam; models expressam dados/regras; templates apresentam.
SQLite e ORM como persistência	Banco único previsto para os ambientes local e de produção (Artefato 9).	Persistência via ORM e arquivo SQLite; produção exige dados persistentes e backup, tratados no Artefato 9.
Relacionamentos explícitos do domínio	Projetos, backlogs, épicos e histórias dependem de associações consistentes.	OneToOne e ForeignKey expressam vínculos e participam da integridade.
Papéis dono e membro por projeto	O Artefato 4 diferencia gestão exclusiva do projeto de edição compartilhada dos requisitos.	Vínculo usuário–projeto com papel de dono ou membro; verificação em cada operação, independente do login.
Autenticação integrada ao framework	Recursos de identidade e sessões já habilitados no Django.	Autenticação é mecanismo transversal; autorização por projeto não deve ser confundida com login.
Estimativa e priorização independentes	Story Points e Effort atendem decisões diferentes: planejamento da Sprint e ordenação do Product Backlog.	Histórias mantêm os valores separados; RICE usa apenas Effort, evitando acoplamento à estimativa da Sprint.
Sem envio automático de e-mail	No Artefato 4, membros são adicionados pelo e-mail de contas existentes; o Artefato 9 exclui serviço de e-mail.	Busca interna por conta cadastrada, sem integração com serviço de mensagens.

As principais **restrições** são: autenticação e autorização por projeto; um Product Backlog por projeto; uma história em um backlog por vez; critérios individuais por história; Story Points e Effort independentes; e armazenamento SQLite nos ambientes previstos.

7 Perspectivas da arquitetura

Perspectiva lógica. A Figura 1 mostra responsabilidades dos componentes MVT e a comunicação com o armazenamento. A Figura 2 apresenta o núcleo de classes do domínio, enquanto a Figura 3 apresenta os vínculos de participação e a distinção conceitual entre estimativa e prioridade. As diferenças entre implementação e especificação estão registradas no texto.

Perspectiva de desenvolvimento. No projeto, a organização permite localizar configurações, dados e mecanismos de entrada sem distribuir o domínio por serviços independentes.

Perspectiva dinâmica/de processos. Fluxo de uma operação: navegador → (Nginx/Gunicorn em produção) → URLconf/middlewares → view → model/ORM → SQLite; resposta via view/template. Ao criar projeto, associa-se dono e cria-se Product Backlog; antes de alterações, verifica-se dono ou membro. Movimentações de histórias mantêm um único backlog e o cálculo RICE usa Effort. As views analisadas também cobrem Sprint Backlogs, histórias, critérios de aceitação, épicos e movimentação entre backlogs.

Perspectiva física abstrata. No desenvolvimento, navegador e servidor Django podem operar

localmente. O navegador acessa HTTPS, Nginx encaminha a Gunicorn/Django e o SQLite reside em armazenamento persistente com backup. Aqui se descrevem conexões entre componentes;

8 Impacto das ferramentas, frameworks e templates

Ferramenta	Impacto na arquitetura	Natureza
Python	Linguagem de implementação dos componentes Django e das regras registradas nos models.	Execução.
Django	Define MVT, roteamento e middlewares; autenticação e sessões identificam cada participante, mas a autorização por projeto é regra da aplicação.	Framework.
Django ORM/migrations	Abstrai o SQL de uso corrente e relaciona classes a tabelas; versiona modificações estruturais.	Persistência.
Django Templates	Permite montar respostas HTML no lado do servidor e mantém apresentação separada do domínio; utilizado nas telas de autenticação, projetos, membros e Product Backlog.	Template web.
SQLite	Banco relacional em arquivo usado localmente e em produção, com persistência e cópias de segurança definidas posteriormente.	Banco.
Nginx/Gunicorn	Em produção, fornecem entrada HTTPS/proxy e execução WSGI da aplicação Django, sem alterar o modelo MVT.	Implantação.
Git/GitHub	Versionam código e documentos em repositório compartilhado; não participam do fluxo da aplicação.	Suporte.

9 Referências da arquitetura

- Universidade de Brasília. *Engenharia de Software – Trabalho Prático.* Requisitos do sistema (p. 1), instruções 15–16 (p. 2) e critérios da atividade 05 (p. 3).